IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

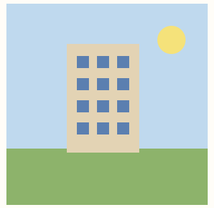
No empieces de cero: reutiliza una red

Teoría, práctica y, al final, material de consulta

Sprint 11 · S11_02 · teoría breve (25 min el 19/11), 75 min de práctica en clase y material de consulta

LA IDEA · EL PROBLEMA

Pocas fotos etiquetadas: no empieces de cero



hotel

pocas fotos



apartamentos

pocas fotos



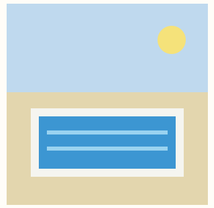
casa rural

pocas fotos



camping

pocas fotos



piscina

pocas fotos

Desde cero

La red tiene que aprender a **mirar** y a **decidir** solo con tus pocas fotos: tiende a **memorizar**.

Reutilizando

Se parte de una red que **ya sabe mirar** y solo se le enseña lo nuevo: hacen falta menos fotos.

Bordes, texturas y formas sirven para casi cualquier imagen: se pueden reutilizar.

LA IDEA · BASE Y CABEZA

Una red tiene base y cabeza

Imagen
32 × 32 × 3

→

BASE · se reutiliza
2 bloques **Conv** → **Pool**
8 · 16 filtros
Extrae rasgos: bordes, texturas, formas

→

Flatten
1 024 rasgos

→

CABEZA · se cambia
Dense (3, softmax)
Decide la clase

1 392
parámetros de la base (red de juguete)

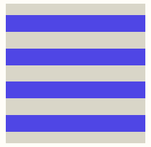
3 075
parámetros de la cabeza (3 clases)

4 467
parámetros en total

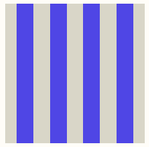
La base aprende a mirar; la cabeza aprende qué hay en tu problema.

LA IDEA · PREENTRENAR

Preentrenar la base en una tarea parecida



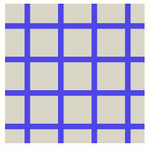
rayas horizontales



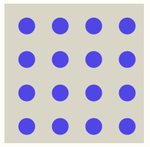
rayas verticales



diagonales



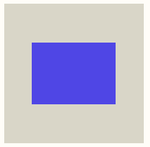
cuadrícula



puntos



círculo



rectángulo



triángulo

En la vida real: ImageNet

Se descarga una base ya entrenada con **millones de fotos** de objetos reales.

En el cuaderno, sin internet

La base aprende antes una tarea auxiliar parecida: reconocer **formas y texturas**.

Qué aprende la base

Bordes, esquinas y patrones repetidos: rasgos universales.

Bordes, esquinas y patrones repetidos también sirven para reconocer edificios y tiendas de campaña.

LA IDEA · CONGELAR LA BASE

Congelar la base y entrenar solo la cabeza

BASE · congelada
trainable = False
sus pesos no cambian

→

CABEZA nueva · se entrena
una capa Dense para tus 5 clases

Estrategia (red de juguete)	Parámetros que se entrenan	Fotos que necesita
Todo desde cero	4 467 (todos)	muchas
Transfer: base congelada	3 075 (solo la cabeza)	**pocas**

Ojo: se congela una **copia** de la base, no la original: así la original se puede reutilizar.

LA IDEA · FINE-TUNING

Fine-tuning: ajustar el último bloque

Fase	Bloque 1	Bloque 2	Bloque 3	Cabeza	Tasa de aprendizaje
1 · Solo la cabeza	congelado	congelado	congelado	**se entrena**	0,001
2 · Fine-tuning	congelado	congelado	**se entrena**	**se entrena**	0,0001

Regla de oro

Primero la cabeza, después el fine-tuning, y con una tasa **10 veces menor**: se ajustan los rasgos más específicos sin destrozar los generales.

Error típico

Cambiar **trainable** y **no recompilar**: el cambio no surte efecto. Usa **compile** otra vez.

Con muy pocas imágenes un entrenamiento suelto es muy variable: se repite con varias semillas.

PRÁCTICA · CUADERNO S11_02

Lo que haremos en el cuaderno

Parte	Qué haremos	Dónde
Preparación	Cargar herramientas, generador y datos	Clase
Ejemplo	CNN desde cero y preentrenar la base	Clase
Ejercicio 1	Congelar la base	Clase
Ejemplo	Entrenar solo la cabeza	Clase
Ejercicio 2	Descongelar solo un bloque	Casa
Para leer	Fine-tuning y tres semillas	Casa
Para leer	MobileNetV2 y rama de Colab	Casa
Mini-reto	¿Cuántas fotos hay que etiquetar?	Casa

Cada ejercicio: **lee** → **escribe** → **ejecuta** → **comprueba el ✔**. La solución está plegada: ábrela tras intentarlo.

CONSULTA

Material de consulta: código y aclaraciones

Cuándo abrirlo

Después de intentar el ejercicio, o cuando algo no te salga. En clase no se proyecta.

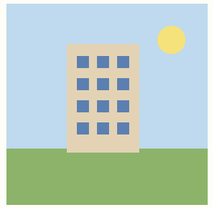
Cómo está ordenado

En el **mismo orden que el cuaderno**, cada ejercicio con su código y sus aclaraciones.

Ejercicio	Tema	Diapositivas
Ejemplo	Punto de partida y preentrenar	9 y 10
1	Congelar la base	11 y 12
Ejemplo	Entrenar solo la cabeza	13
2	Descongelar un bloque	14 y 15
Ejemplo	Tres semillas y MobileNetV2	16 y 17
Mini-reto	¿Cuántas fotos?	18 y 19

CONSULTA · SECCIONES 0 A 2 · EL PUNTO DE PARTIDA

Solo 6 imágenes por tipo de alojamiento



hotel

6 imágenes



apartamentos

6 imágenes



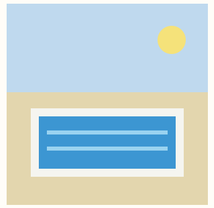
casa rural

6 imágenes



camping

6 imágenes



piscina

6 imágenes

30

para entrenar: 5 clases × 6, con mucho ruido

200 · 300

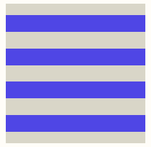
para validar · para probar

0,800

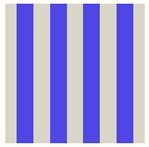
exactitud en prueba de una CNN **desde cero**: la referencia

Son dibujos esquemáticos, no fotos reales: sirven para ver el método completo sin internet.

Preentrenar la base en una tarea parecida



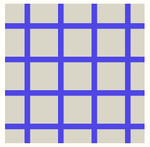
rayas horizontales



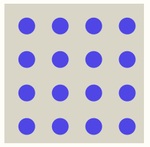
rayas verticales



diagonales



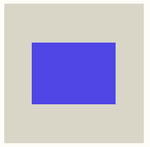
cuadrícula



puntos



círculo



rectángulo



triángulo

En la vida real: ImageNet

Se descarga una base ya entrenada con **millones de fotos** de objetos reales.

En el cuaderno, sin internet

La base aprende antes **8 clases de formas y texturas**: 3 200 imágenes (400 por clase).

0,938

exactitud de la base en la tarea auxiliar (validación)

Bordes, esquinas y patrones repetidos también sirven para reconocer edificios y tiendas de campaña.

CONSULTA · EJERCICIO 1

Congelar la base (ejercicio 1)

```
def congelar(modelo):
    modelo.trainable = False
    return modelo
copia_base = congelar(copiar_base(base_pre, "base_congelada"))
len(copia_base.trainable_weights)    # 0
len(base_pre.trainable_weights)      # 6
```

0

pesos entrenables en la **copia**

6

pesos entrenables en la **original**

Se congela una copia: la base original queda libre para reutilizarla.

CONSULTA · ACLARACIONES · EJERCICIO 1

Leer el congelado (ejercicio 1)

modelo.trainable = False

Pone **todas** las capas como no entrenables: sus pesos no cambian al entrenar. Hay que **devolver** el modelo.

copiar_base(...)

Hace una **copia independiente** con los mismos pesos: se congela la copia y la base original queda libre para otros usos.

Los assert

En la copia **no queda ningún peso entrenable** y todos pasan a «no entrenables»; la original conserva los suyos.

Si falla

Tras congelar no debe quedar ningún peso entrenable → falta **trainable = False**. **TypeError: 'NoneType'...** → falta **return modelo**.

Entrenar solo la cabeza (base congelada)

BASE · congelada
trainable = False
sus pesos no cambian

→

CABEZA nueva · se entrena
una capa Dense para tus 5 clases

Estrategia (30 imágenes)	Parámetros que se entrenan	Exactitud en prueba
CNN desde cero	35 109 (todos)	0,800
Transfer: base congelada	11 525 (solo la cabeza)	**0,833**

Con un solo entrenamiento la diferencia es pequeña: se comprueba con **varias semillas.**

CONSULTA · EJERCICIO 2

Descongelar un bloque (ejercicio 2)

```
def descongelar_bloque(base, n):
    base.trainable = True
    for capa in base.layers:
        capa.trainable = capa.name.startswith(f"bloque{n}")
    return len(base.trainable_weights)
descongelar_bloque(copiar_base(base_pre, "prueba"), 3)    # 2
```

2

pesos entrenables del bloque 3

0,830

exactitud tras el fine-tuning (antes, 0,833)

Primero la cabeza; después, el último bloque, con tasa pequeña (1e-4) y recompilando.

CONSULTA · ACLARACIONES · EJERCICIO 2

Leer el descongelado (ejercicio 2)

capa.trainable = capa.name.startswith(...)

Solo las capas del bloque **n** quedan **entrenables**; las demás, congeladas. Se reconocen por el nombre (**bloque3_conv**).

return len(base.trainable_weights)

Cuántos pesos quedan entrenables: **2** por bloque (filtros y sesgos de su convolución).

Después, recompilar

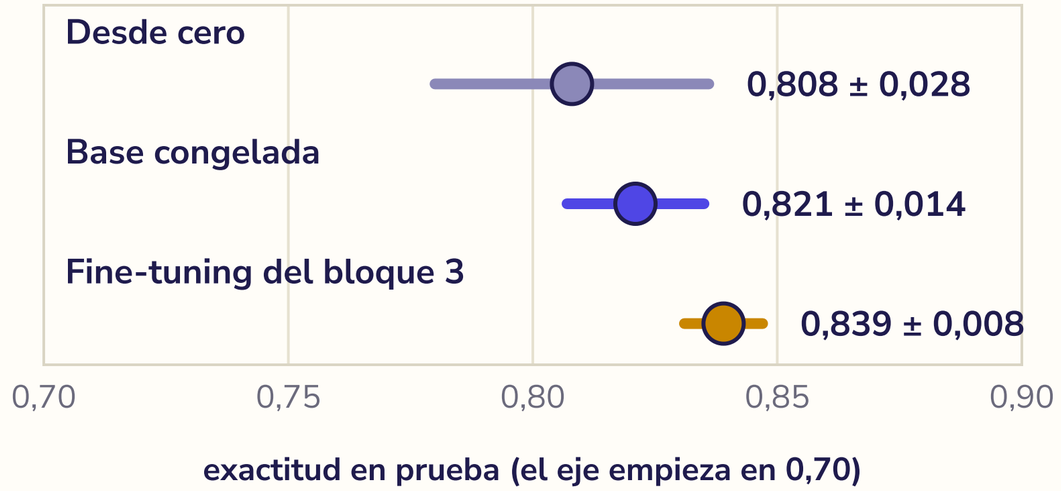
Al cambiar qué es entrenable (y bajar la tasa a **1e-4**) hay que llamar otra vez a **compile**: si no, el cambio no surte efecto.

Un solo experimento

0,833 → **0,830**: no mejora. Con 30 imágenes **un entrenamiento suelto no decide**: se repite con varias semillas.

Tres estrategias, tres semillas

Media ± desviación de 3 semillas · 30 imágenes de entrenamiento



Cómo leerlo

Con pocas imágenes, la base preentrenada ayuda: **0,808** → **0,839** con fine-tuning. Pero la mejora es **modesta** y los rangos se solapan.

Qué aprender

El método importa más que la cifra. **No** se promete que el transfer learning gane siempre.

CONSULTA · SECCIÓN 6 · MOBILENETV2

MobileNetV2: la misma estructura

MobileNetV2 con ImageNet	Qué cambia
Base	ya entrenada con millones de fotos reales
Preprocesado	el que pide la red (**preprocess_input**)
Tamaño de entrada	se reescalan las imágenes a lo que pide la red
Batch norm	al congelar la base: **training=False**

Sin internet la celda usa pesos aleatorios; los de ImageNet se prueban en Colab y no están verificados en el taller.

154 · 416 613

capas de la base · parámetros totales (pesos aleatorios, sin internet)

6 405

parámetros entrenables: solo la cabeza

CONSULTA · MINI-RETO

¿Cuántas fotos hay que etiquetar?

```
curva_datos = {}
for n in (3, 6, 12, 24):
    Xn, yn = generar_lote(alojamiento, CLASES_ALOJ, n, semilla=20)
    keras.utils.set_random_seed(SEMILLA)
    m_cero = con_cabeza(construir_base("cero"), 5); entrenar(m_cero, 40, SEMILLA, Xn, yn)
    m_tr = con_cabeza(congelar(copiar_base(base_pre, "b")), 5); entrenar(m_tr, 30, SEMILLA, Xn, yn)
    curva_datos[n] = (m_cero.evaluate(X_pru, y_pru, verbose=0)[1], m_tr.evaluate(X_pru, y_pru, verbose=0)[1])
```

Fotos por clase	3	6	12	24
Desde cero	0,737	0,843	0,917	0,977
Transfer (base congelada)	**0,817**	**0,873**	**0,957**	0,977

El transfer learning gana con **pocas** fotos; con 24 por clase, la ventaja desaparece.

CONSULTA · ACLARACIONES · MINI-RETO

Leer el mini-reto

generar_lote(..., n, semilla=20)

Genera un entrenamiento con **n imágenes por clase** (3, 6, 12 y 24). La prueba es siempre la misma: la comparación es justa.

Dos modelos por cada n

Desde cero (**40 épocas**) y con la base preentrenada congelada (**30**). Se guarda la pareja de exactitudes.

Con 3 por clase

Transfer **0,817** frente a **0,737** desde cero: unos **8 puntos** más.

Con 24 por clase

La ventaja **desaparece**: las dos acaban en **0,977**. Con 12, el transfer ya da 0,957.